



Estimate with FIMS : : CHEATSHEET

GET FIMS

```
install.packages("FIMS", repos = c("https://noaa-fisheries-integrated-toolbox.r-universe.dev", "https://cloud.r-project.org"))
```

TIDY THE INPUT FIMSFrame

Prepare one tidy data frame for your FIMS model

```
model_inputs <-  
FIMSFrame(your_long_data_frame)
```

Investigate the FIMSFrame class

get_*() functions extract information from a FIMSFrame object

```
data("data_big", package = "FIMS")  
data_4_model <- FIMSFrame(data_big)  
get_n_fleets(data_4_model)  
[1] 2  
get_ages(data_4_model)  
[1] 1 2 3 4 5 6 7 8 9 10 11 12  
get_data(data_4_model)
```

Package data

Structure of the long, data format

type	fleet	timing	value	unit	uncertainty	age
landings	trawl	2025	1000	mt	"~dlnorm(meanlog = log_landings_expected, sdlog = 0.0001)"	NA
landings	other_fleet	2024	2366	mt	"~dlnorm(meanlog = log_landings_expected, sdlog = 0.0001)"	NA
age_comp	trawl	2025	0.1	proportion	"~dmultinom(prob = agecomp_proportion, size = 100)"	1
age_comp	trawl	2025	0.2	proportion	"~dmultinom(prob = agecomp_proportion, size = 100)"	2
...	...	...	...	...	...	...

type: landings, index, age_comp, length_comp, weight_at_age, age_to_length_conversion
fleet: name of the fleet, can be any string
timing: double values specifying the collection timing
value: the measurement
unit: unit of the measurement
uncertainty: formula specify the distribution and parameters for that distribution
age: desired binning structures for your model, can have additional binning columns such as **length** and, in the future, **sex**

FIT A MODEL FIMSFit

Default parameters based on data

```
fit <- setup_default_parameters(data = model_inputs) |>  
  initialize_fims(  
    parameters = parameters,  
    data = model_inputs  
  ) |>  
  fit_fims(optimize = TRUE)
```

Send all of the specified parameters and data to their respective C++ modules

Optimize the model using nlminb as the default

Default model configurations are created based on the data, e.g., logistic selectivity for each fleet in your data

See [vignette](#) for details on modifying the parameters tibble

Investigate the FIMSFit class get_*() functions extracts information from a FIMSFit object

```
get_estimates(fit)  
get_number_of_parameters(fit)  
get_run_time(fit)
```

```
# TMB-specific information  
get_report(fit)  
get_obj(fit)  
get_par(fit)
```



Include Code

CODE CELLS

Code cells start with ````{language}` and end with `````.

  Use **Insert Code Chunk/Cell**

```
```{r}
#| label: chunk-id
library(tidyverse)
```
```

```
```{python}
#| label: chunk-id
import pandas as pd
```
```

Other languages: {julia}, {ojs}

Add code cell options with `#|` comments.

Cell options control **execution**, **figures**, **tables**, **layout** and more. See them all at: <https://quarto.org/docs/reference/cells>

EXECUTION OPTIONS

OPTION DEFAULT EFFECTS

| | | |
|----------------|-------|---|
| echo | true | false: hide code fenced: include code cell syntax |
| eval | true | false: don't run code |
| include | true | false: don't include code or results |
| output | true | false: don't include results
asis: treat results as raw markdown |
| warning | true | false: don't include warnings in output |
| error | false | true: include error in output and continue with render |

Set execution options at the **cell level**:

```
```{r}
#| echo: false
```
```

```
```{python}
#| echo: false
```
```

Or, **globally** in the YAML header with the **execute** option:

```
---
execute:
  echo: false
---
```

Set options in code cells with `#|` comments and **YAML syntax**:
key: value

INLINE CODE

Use computed values directly in text sections. Code is evaluated at render and results appear as text.

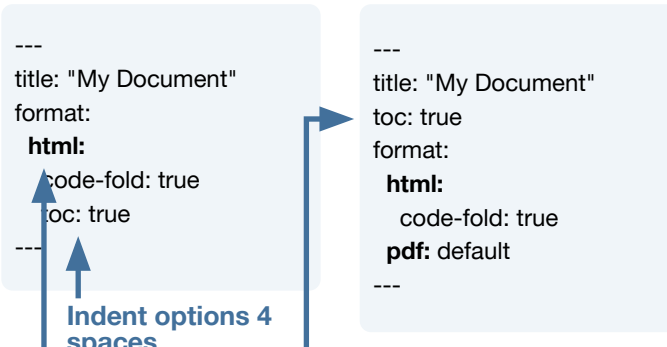
KNITR **JUPYTER** **OUTPUT**

Value is ``r 2 + 2``. Value is ``{python} 2 + 4``. Value is 4.



Set Format and Options

SET FORMAT OPTIONS MULTIPLE FORMATS



Common formats: **html**, **pdf**, **docx**, **odt**, **rtf**, **gfm**, **pptx**, **revealjs**, **beamer**

Render **all** formats:

Terminal

```
quarto render hello.qmd
```

Render a **specific** format:

Terminal

```
quarto render hello.qmd --to pdf
```

Add Content

FIGURES

MARKDOWN

```
![[CAP]](image.png){#fig-LABEL fig-alt="ALT"}
```

COMPUTATION

```
```{python}
#| label: fig-LABEL
#| fig-cap: CAP
#| fig-alt: ALT
{{ plot code here }}
```
```

Or {r}

You can see in `@fig-scatterplot`, that...

| Prefix | Renders | Prefix | Renders |
|--------|----------|--------|------------|
| fig- | Figure 1 | eq- | Equation 1 |
| tbl- | Table 1 | sec- | Section 1 |

OPTION

html/revealjs
pdf/beamer
docx/pptx

DESCRIPTION

| | | | |
|---------|------------------------------|-------|---|
| Nav | toc | X X X | Add a table of contents (true or false) |
| | toc-depth | X X X | Lowest level of headings to add to table of contents (e.g. 2, 3) |
| | anchor-sections | X | Show section anchors on mouse hover (true or false) |
| Style | highlight-style | X X X | Syntax highlighting theme (e.g. arrow, pygments, kate, zenburn) |
| | mainfont, monofont | X X | Font name. HTML: sets CSS font-family; LaTeX: via fontspec package |
| | theme | X | Bootswatch theme name (e.g. cosmo, darkly, solar etc.) |
| LaTeX | css | X | CSS or SCSS file to use to style the document (e.g. "style.css") |
| | reference-doc | X | docx/pptx file containing template styles (e.g. file.docx, file.pptx) |
| | include-in-header | X X | Files of content to include in header of output document, also include-before-body , include-after-body |
| Code | keep-md | X X X | Keep intermediate markdown (true or false), also keep-ipynb , keep-tex |
| | documentclass | X | LaTeX document class, set document class options with classoption |
| | pdf-engine | X | LaTeX engine to produce PDF output (xelatex, pdflatex, lualatex) |
| Figures | cite-method | X | Method used to format citations (citeproc, natbib, biblatex) |
| | code-fold | X | Let readers toggle the display of R code (false, true, or show) |
| | code-tools | X | Add menu for hiding, showing, and downloading code (true or false) |
| Figures | code-overflow | X | Display of wide code (scroll, or wrap) |
| | fig-align | X X / | Alignment of figures (default, left, right, or center) |
| | fig-width, fig-height | X X X | Default width and height for figures in inches |
| Figures | fig-format | X X X | Format for Matplotlib or R figures (retina, png, jpeg, svg, or pdf) |

Also use in code cells

TABLES

MARKDOWN

```
object	radius
Sun	696000
Earth	6371

: CAPTION {#tbl-LABEL}
```



COMPUTATION Output a Markdown table or an HTML table from your code

KNITR

Use `knitr::kable()` to produce Markdown:

```
```{r}
#| label: tbl-LABEL
#| tbl-cap: CAPTION
knitr::kable(head(cars))
```
```

Also see the R packages: `gt`, `flextable`, `kableExtra`.

JUPYTER Add Markdown() to Markdown output:

```
```{python}
#| label: tbl-LABEL
#| tbl-cap: CAPTION
import pandas as pd, tabulate
from IPython.display import Markdown
df = pd.DataFrame({"A": [1, 2],
 "B": [1, 2]})
Markdown(df.to_markdown(index=False))
```
```

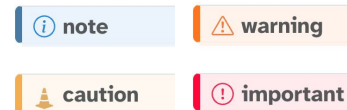
CALLOUTS

tip

```
... {callout-tip}
## Title
```

Text
...

Instead of **tip** use one of:
note, caution, warning, or important.



```
---
bibliography: references.bib
---
```



Build your bibliography file from your Zotero library, DOI, Crossref, DataCite, or PubMed